



GlobalCommerce Enterprise Platform

Cross-border commerce
Inventory and logistics
Demand forecasting

Presented by:
Jawad Ahmad
Diploma in AIOPS
EduQual Level 6

The business problem

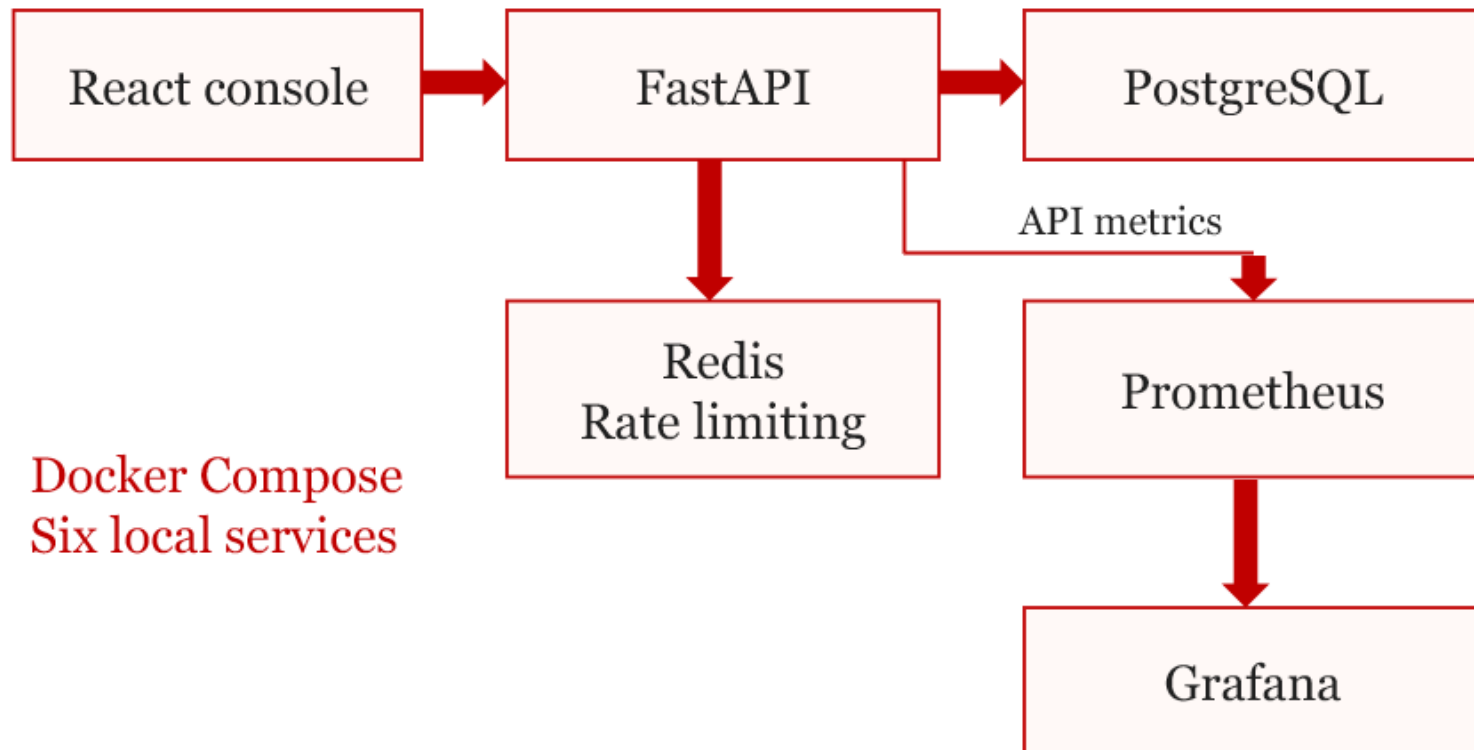
- Separate systems make international orders difficult to coordinate.
- Stock reservations and shipment updates must stay connected.
- Demand uncertainty creates stockout and overstock risk.
- One workspace gives operations teams a shared view of each order.

What the prototype covers

- Marketplace: merchant, customer and product records.
- Operations: warehouse stock, international orders and shipment events.
- Decision support: demand forecasts and replenishment recommendations.
- Visibility: commerce views, Prometheus metrics and Grafana dashboards.

System architecture

A modular application with shared data and separate monitoring services.



Docker Compose
Six local services

Forecasting runs
inside FastAPI.

PostgreSQL stores
business records.

*Simple to run locally.
Clear boundaries to evolve.*

Inventory you can explain

Available stock excludes units already promised to existing orders.

$$\begin{array}{rcccl} \mathbf{120} & - & \mathbf{3} & = & \mathbf{117} \\ \text{On hand} & & \text{Reserved} & & \text{Available} \end{array}$$

Illustrative stock balance

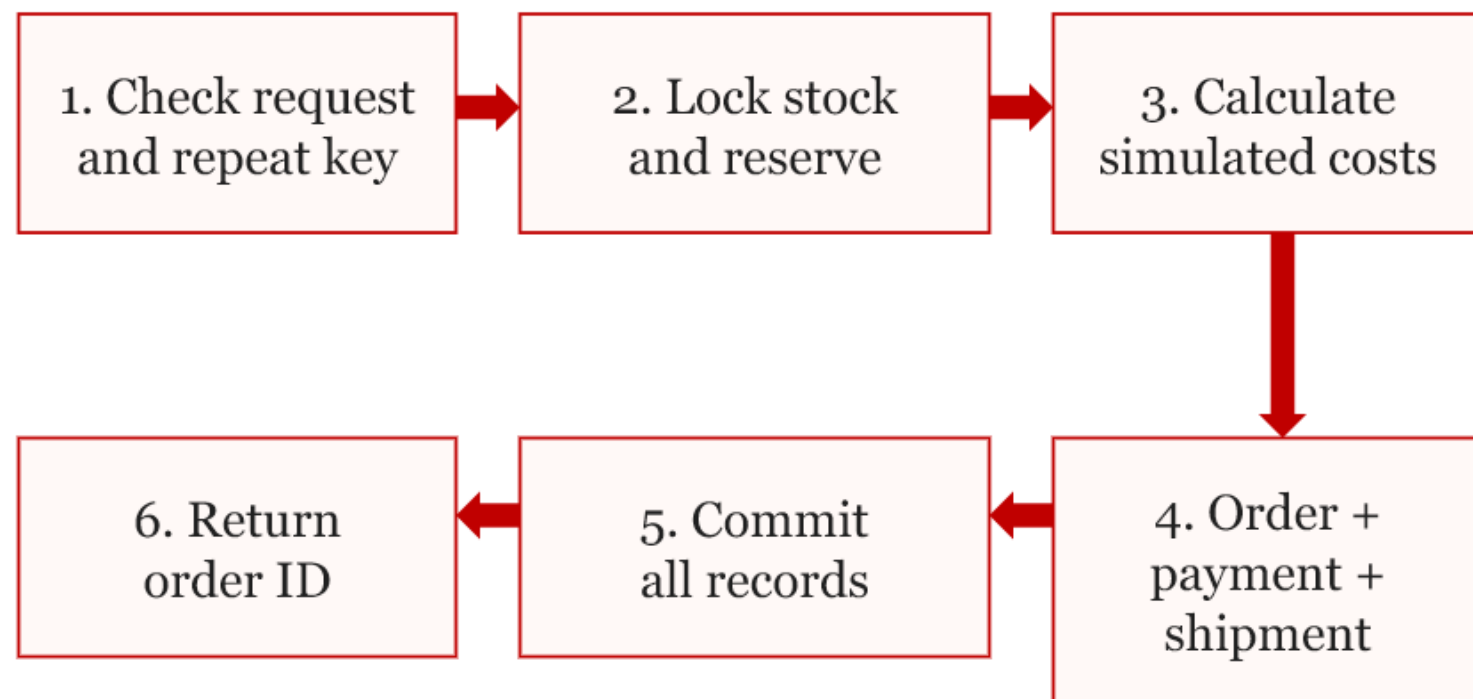
Stock is tracked for each product and warehouse.

Reservation is not a physical shipment.

*Reserve at order time.
Deduct at delivery.*

One order, one workflow

Business records are created together; failures should not leave partial orders.



Successful creation commits together.

An error should roll back changes.

A repeat key returns the existing order.

International order costing

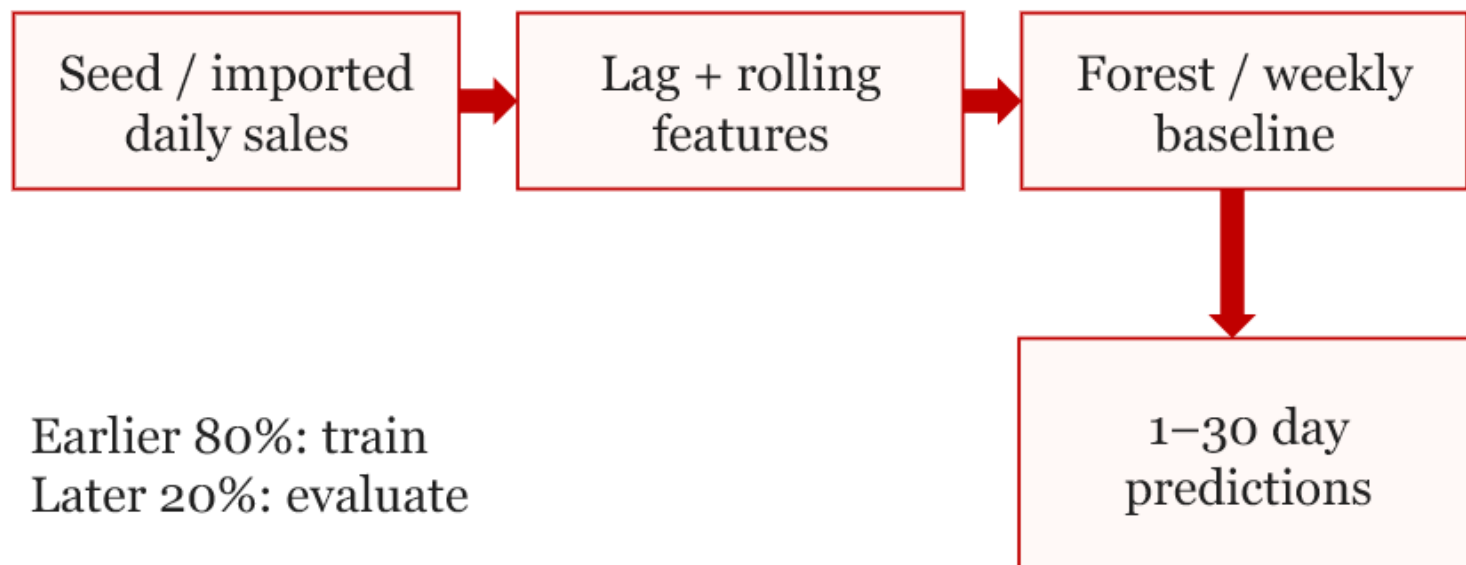
- Supported currencies: USD, GBP, AED, PKR and EUR.
- Order total combines goods, shipping, duty and tax.
- Exchange rates, payment authorization and trade charges are simulated.
- Real launch requires current rates, legal rules and provider integrations.

Logistics visibility

- Each order links to a shipment and tracking reference.
- Events record status, location, timestamp and a note.
- The server enforces progress, delay and recovery rules.
- Delivery captures payment and deducts stock once.

Demand forecasting

Past sales become features, predictions and a replenishment signal.



MAE / RMSE
measure holdout error.

Intervals are
residual-based,
not calibrated.

*Evaluate on later dates.
Do not shuffle time.*

From forecast to stock

Suggested replenishment covers predicted demand plus a safety buffer.

$$60 + 15 - 50 = 25$$

Demand + safety stock – available stock

Suggested order: 25 units

Illustrative seven-day example; result cannot be below zero.

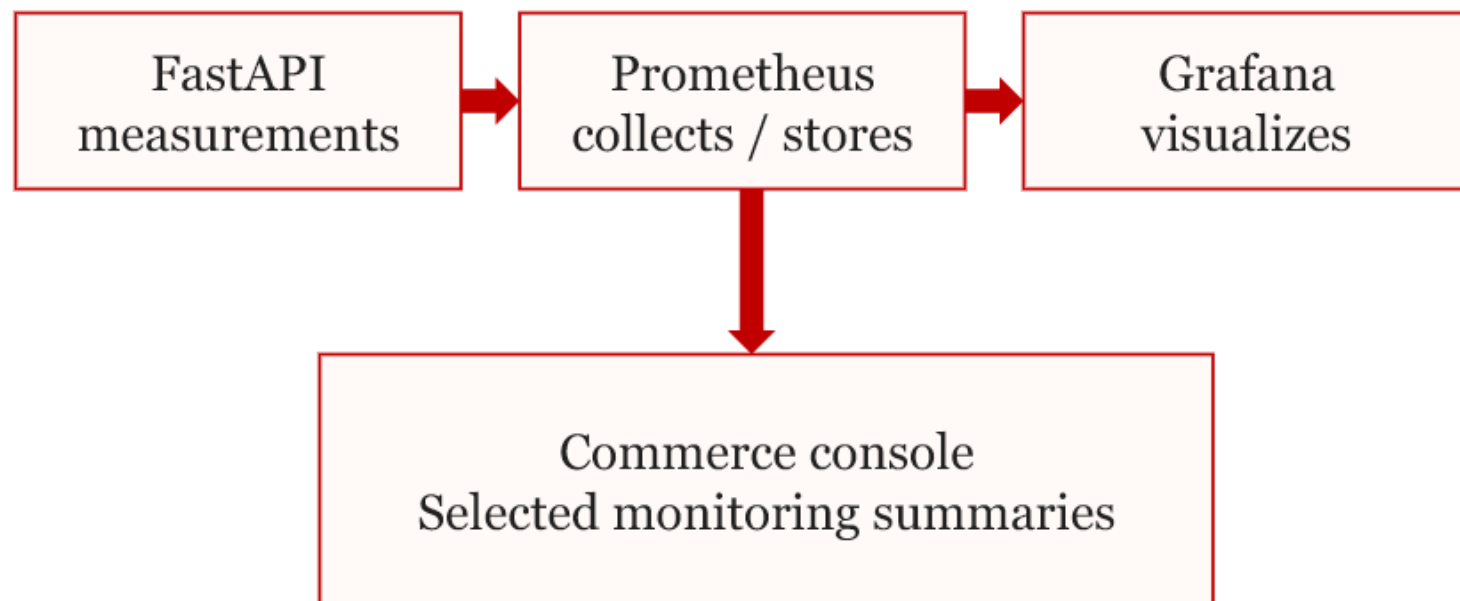
A recommendation
needs review.

Supplier lead times
and purchasing are
future extensions.

*Decision support.
No automatic purchase.*

Monitoring has three roles

The console summarizes operations; monitoring explains service behavior.



Business view:
orders and stock

Technical view:
traffic, latency
and errors

*One front door.
Different responsibilities.*

The local operating workspace

GlobalCommerce

INVENTORY · GLOBAL OPERATIONS

Inventory

[Refresh](#) [New order](#)

FULFILMENT NETWORK

Inventory by warehouse

PRODUCT	WAREHOUSE	ON HAND	RESERVED	AVAILABLE	HEALTH
Connected Home Control Hub NT-HUB-03	LON-01 London	110	0	110	● HEALTHY
Pulse Pro Smart Watch NT-WATCH-02	LON-01 London	34	0	34	● REORDER
Adaptive Noise-Cancelling Headphones NT-NOISE-01	LON-01 London	68	0	68	● HEALTHY
Connected Home Control Hub NT-HUB-03	DXB-01 Dubai	180	3	177	● HEALTHY
Pulse Pro Smart Watch NT-WATCH-02	DXB-01 Dubai	75	0	75	● HEALTHY
Adaptive Noise-Cancelling Headphones NT-NOISE-01	DXB-01 Dubai	120	0	120	● HEALTHY
Connected Home Control Hub NT-HUB-03	KHI-01 Karachi	95	0	95	● HEALTHY
Pulse Pro Smart Watch	KHI-01				

PLATFORM STATUS
● Operational
[Sign out](#)

Stock by warehouse
On hand – reserved
= available

Security and accountability

- JWT authentication, Argon2 hashes and active-account checks.
- Server-side role and customer/merchant ownership rules.
- Rate limiting, request IDs, audit records and stock locks.
- Localhost only; production still needs MFA, TLS and managed secrets.

Evidence before confidence

- 20 backend regression tests; about 90% application line coverage.
- PostgreSQL checks: duplicate requests, competing stock and delivery.
- Frontend contracts, TypeScript checks and production build.
- Dated security reports; no production load or real-data accuracy claim.

Live demonstration

- 1. Show services, sign in, and inspect a warehouse stock balance.
- 2. Create a two-unit GBP order; confirm the reservation and charges.
- 3. Advance its shipment; show the persisted event history.
- 4. Run a seven-day forecast; inspect Monitoring and Grafana.

From prototype to production

- Integrate real payments, carriers, exchange and trade rules.
- Add TLS, MFA, managed secrets and restricted staff scopes.
- Validate real sales with rolling backtests and calibrated ranges.
- Add workers, split shipments, returns, procurement and load tests.

Conclusion

- One connected journey: inventory, order, shipment and demand planning.
- A reproducible local stack with visible business and technical records.
- A clear distinction between working features and production readiness.
- Thank you. I welcome your questions and design challenges.

Appendix: design questions

- Why not microservices? Local simplicity; split only when a real need appears.
- How to avoid overselling? Transactional reservations plus concurrency tests.
- Why not trust one accuracy score? Horizon, baseline and data quality matter.
- What happens on failure? Roll back, return a clear error, then retry safely.

Appendix: evidence and tools

- Assessment sources: supplied exam guide, assigned topic and sample deck.
- Implementation evidence: source code, Compose configuration and test suite.
- AI tools assisted code, documentation and presentation preparation.
- The viva must demonstrate the student's own understanding and verification.